Отчёт по выполнению лабораторной работы №2

Операционные системы

Головко Екатерина Андреевна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **9**

4.1 Установка программного обеспечения 9
4.2 Базовая настройка git 9
4.3 Создайте ключи ssh 10
4.4 Создайте ключи pgp 11
4.5 Настройка github 12
4.6 Добавление PGP ключа в GitHub 12
4.7 Настройка автоматических подписей коммитов git 13
4.8 Настройка gh 13
4.9 Настройка рабочего пространства и репозитория 14

5 Выводы **16**

Список иллюстраций

4.1 Установка git 9
4.2 Установка gh 9
4.3 Имя и почта владельца репозитория 9
4.4 Настройка utf-8 10
4.5 Имя начальной ветки 10
4.6 параметр autocrlf и safecrlf 10
4.7 Создание ssh-ключа по алгоритму rsa 10
4.8 Создание ssh-ключа по алгоритму ed25519 11
4.9 Создание pgp-ключа 11
4.10 Вывод списка ключей 12
4.11 Копирование ключа 12
4.12 Добавление ключа 13
4.13 Настройка автоматических подписей коммитов 13
4.14 Авторизация в gh 13
4.15 Завершение авторизации 14
4.16 Создание шаблона 14
4.17 Создание шаблона 14
4.18 Создание шаблона 14
4.19 Переход в каталог и удаление лишних файлов 15
4.20 Создание каталогов 15
4.21 Отправка файлов на сервер 15

Список таблиц

1 Цель работы

Изучить идеологию и применение средств контроля версий и освоить умения по работе с git.

2 Задание

1.Установка программного обеспечения 2.Базовая настройка git 3.Создайте ключи ssh 4.Создайте ключи pgp 5.Настройка github 6.Добавление PGP ключа в GitHub 7.Настройка автоматических подписей коммитов git 8.Настройка gh 9.Настройка рабочего пространства и репозитория

3 Теоретическое введение

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется. В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений, пользователь размещает новую версию в хранилище. При этом предыдущие версии не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять не полную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных. Системы контроля версий поддерживают возможность отслеживания и разрешения конфликтов, которые могут возникнуть при работе нескольких человек над одним файлом. Можно объединить (слить) изменения, сделанные разными участниками (автоматически или вручную), вручную выбрать нужную версию, отменить изменения вовсе

или заблокировать файлы для изменения. В зависимости от настроек блокировка не позволяет другим пользователям получить рабочую копию или препятствует изменению рабочей копии файла средствами файловой системы ОС, обеспечивая таким образом, привилегированный доступ только одному пользователю, работающему с файлом. Системы контроля версий также могут обеспечивать дополнительные, более гибкие функциональные возможности. Например, они могут поддерживать работу с несколькими версиями одного файла, сохраняя общую историю изменений до точки ветвления версий и собственные истории изменений каждой ветви. Кроме того, обычно доступна информация о том, кто из участников, когда и какие изменения вносил. Обычно такого рода информация хранится в журнале изменений, доступ к которому можно ограничить. В отличие от классических, в распределённых системах контроля версий центральный репозиторий не является обязательным. Среди классических VCS наиболее известны CVS, Subversion, а среди распределённых — Git, Bazaar, Mercurial. Принципы их работы схожи, отличаются они в основном синтаксисом используемых в работе команд.

4 Выполнение лабораторной работы

4.1 Установка программного обеспечения

Устанавливаю git и gh (рис. 4.1,рис. 4.2).

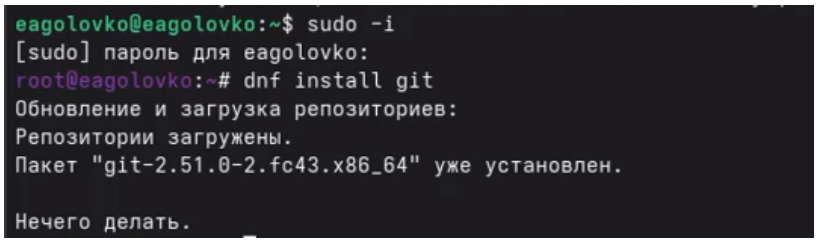

Рисунок 4.1: Установка git

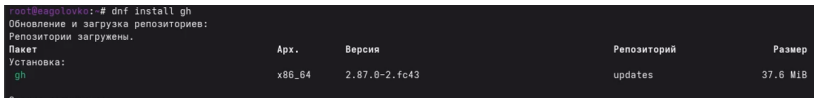

Рисунок 4.2: Установка gh

4.2 Базовая настройка git

Задаю имя и почту владельца репозитория (рис. 4.3).

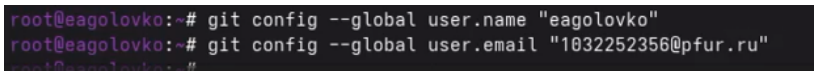

Рисунок 4.3: Имя и почта владельца репозитория

Настроиваю utf-8 в выводе сообщений git (рис. 4.4).

```
root@eagolovko:~# git config --global core.quotepath false
```

Рисунок 4.4: Настройка utf-8

Задаю имя начальной ветки (рис. 4.5).

```
eagolovko@eagolovko:~$ git config --global init.defaultBranch master
```

Рисунок 4.5: Имя начальной ветки

Задаю параметр autocrlf (рис. 4.6).

```
eagolovko@eagolovko:~$ git config --global core.autocrlf input
eagolovko@eagolovko:~$ git config --global core.safecrlf warn
```

Рисунок 4.6: параметр autocrlf и safecrlf

4.3 Создайте ключи ssh

Создаю по алгоритму rsa с ключём размером 4096 бит (рис. 4.7).

```
eagolovko@eagolovko:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/eagolovko/.ssh/id_rsa):
Created directory '/home/eagolovko/.ssh'.
Enter passphrase for "/home/eagolovko/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/eagolovko/.ssh/id_rsa
Your public key has been saved in /home/eagolovko/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:TgFU+BXkNH1+3AseLbh9oFYSizLbSAZ/a5Gcxp4lgEo eagolovko@eagolovko
The key's randomart image is:
+---[RSA 4096]----+
|      .+o..=o     |
|   E o.o o.o. .   |
|  . . o.=.= +o...|
|   .   *.% = *..+|
|      o S * B =..|
|       = * + o o |
|        o .   .  |
|                 |
|                 |
+----[SHA256]-----+
```

Рисунок 4.7: Создание ssh-ключа по алгоритму rsa

Создаю по алгоритму ed25519 (рис. 4.8).

```
eagolovko@eagolovko:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/eagolovko/.ssh/id_ed25519):
Enter passphrase for "/home/eagolovko/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/eagolovko/.ssh/id_ed25519
Your public key has been saved in /home/eagolovko/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:XYH5fNLosxe5gU27xhMTsia+QVVFikup72+iS+yAzJY eagolovko@eagolovko
The key's randomart image is:
+--[ED25519 256]--+
|           o. .oo|
|          o  = . |
|           o*o.  |
|          . =*.=  |
|         S +..O + |
|     o o o..* O  |
|      E ..+o.+ B |
|     .   +.oo O  |
|          =+.*.. |
+----[SHA256]-----+
```

Рисунок 4.8: Создание ssh-ключа по алгоритму ed25519

4.4 Создайте ключи pgp

Создаю ключи pgp (рис. 4.9).

```
eagolovko@eagolovko:~$ gpg --full-generate-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/home/eagolovko/.gnupg'
Выберите тип ключа:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
         0 = не ограничен
      <n>  = срок действия ключа - n дней
      <n>w = срок действия ключа - n недель
      <n>m = срок действия ключа - n месяцев
      <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: eagolovko
Адрес электронной почты: 1032252356@pfur.ru
Примечание:
```

Рисунок 4.9: Создание pgp-ключа

4.5 Настройка github

Аккаунт на гитхаб был создан в прошлом семестре, поэтому не выполняю данный этап лабораторной работы.

4.6 Добавление PGP ключа в GitHub

Вывожу список ключей и копирую отпечаток приватного ключа (рис. 4.10).

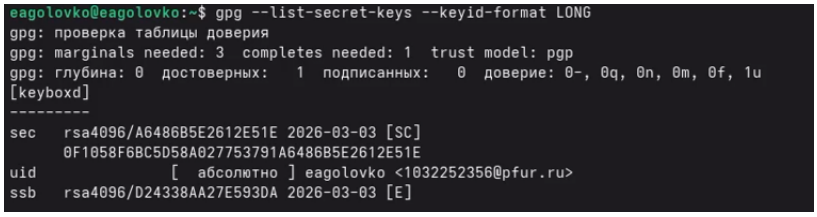

Рисунок 4.10: Вывод списка ключей

Копирую сгенерированный ключ в буфер обмена (рис. 4.11).

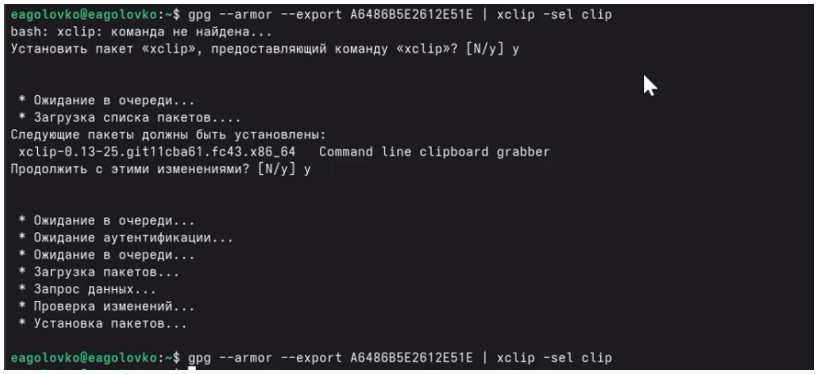

Рисунок 4.11: Копирование ключа

Захожу на сайт githab, перехожу в настройки ключей и добавляю новый ключ (рис. 4.12).

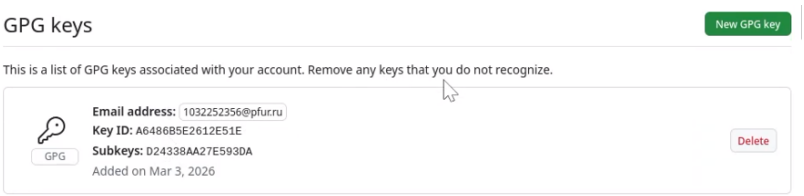

Рисунок 4.12: Добавление ключа

4.7 Настройка автоматических подписей коммитов git

Ввожу команды в терминал, чтобы гит применял указанную почту при создании коммитов (рис. 4.14).

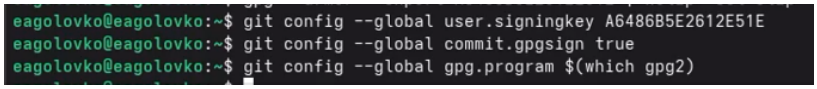

Рисунок 4.13: Настройка автоматических подписей коммитов

4.8 Настройка gh

Авторизовываюсь через терминал (рис. 4.14).

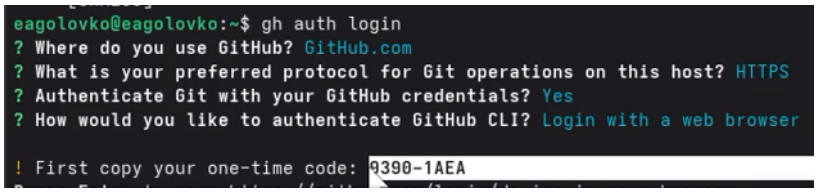

Рисунок 4.14: Авторизация в gh

Затем завершаю настройку через браузер (рис. 4.15).

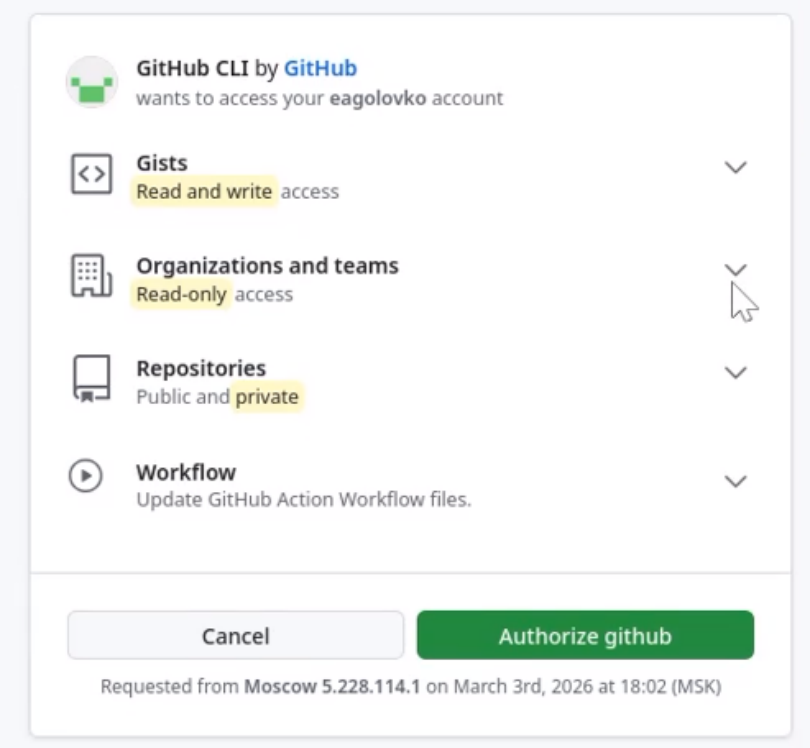

Рисунок 4.15: Завершение авторизации

4.9 Настройка рабочего пространства и репозитория

Создаю шаблон рабочего пространства (рис. 4.16, рис. 4.17, рис. 4.18).

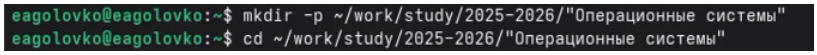

Рисунок 4.16: Создание шаблона

Рисунок 4.17: Создание шаблона

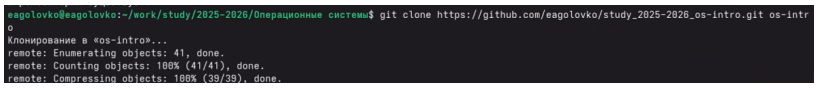

Рисунок 4.18: Создание шаблона

Перехожу в каталог курса и удаляю лишние файлы (рис. 4.19).

```
eagolovko@eagolovko:~/work/study/2025-2026/Операционные системы$ cd os-intro
eagolovko@eagolovko:~/work/study/2025-2026/Операционные системы/os-intro$ rm package.json
```

Рисунок 4.19: Переход в каталог и удаление лишних файлов

Создаю необходимые каталоги (рис. 4.20).

```
eagolovko@eagolovko:~/work/study/2025-2026/Операционные системы/os-intro$ echo os-intro > COURSE
eagolovko@eagolovko:~/work/study/2025-2026/Операционные системы/os-intro$ make
```

Рисунок 4.20: Создание каталогов

Отправляю файлы на сервер (рис. 4.21).

```
eagolovko@eagolovko:~/work/study/2025-2026/Операционные системы/os-intro$ git add .
eagolovko@eagolovko:~/work/study/2025-2026/Операционные системы/os-intro$ git commit -am 'feat(main): make course structure'
[master 7a1f3fd] feat(main): make course structure
 2 files changed, 1 insertion(+), 14 deletions(-)
 delete mode 100644 package.json
eagolovko@eagolovko:~/work/study/2025-2026/Операционные системы/os-intro$ git push
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
При сжатии изменений используется до 6 потоков
Сжатие объектов: 100% (2/2), готово.
Запись объектов: 100% (3/3), 947 байтов | 947.00 КиБ/с, готово.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/eagolovko/study_2025-2026_os-intro.git
   db0b8b0..7a1f3fd  master -> master
```

Рисунок 4.21: Отправка файлов на сервер

5 Выводы

При выполнении данной лабораторной работы я изучила идеологию и применение средств контроля версий, освоила умение по работе с git.